



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

FACULTAD DE INGENIERÍA

Sistemas Operativos

Informe técnico:

Evaluación experimental de Sistema distribuido

Docente:

Marco Antonio López Medina

Alumnos:

Rodrigo Mejía Fabila

Diego Alcantara Ortega

1. Resumen Ejecutivo

El presente informe detalla los resultados obtenidos durante la fase de experimentación y evaluación de una plataforma distribuida de registro de pedidos. La arquitectura fue diseñada e implementada utilizando microservicios desarrollados en Python (FastAPI), una base de datos relacional PostgreSQL, contenerización mediante Docker y orquestación local en un clúster de Kubernetes (Minikube). El objetivo central radica en analizar el rendimiento, la escalabilidad horizontal, la concurrencia bajo estrés y la tolerancia a fallos del sistema, validando métricas como la latencia, solicitudes por segundo (req/s) y tasa de errores.

2. Arquitectura y Configuración del Entorno

El entorno de pruebas operó sobre una máquina Windows 11 con WSL2 (Ubuntu), ejecutando Minikube con el driver de Docker. Los componentes desplegados incluyen:

• API REST (Procesador): Despliegue escalable desde 1 hasta 3 réplicas, expuesto vía ClusterIP Service.

• Persistencia: Un pod individual de PostgreSQL (versión 15-alpine), configurado mediante Secrets y ConfigMaps, con almacenamiento efímero interno al pod.

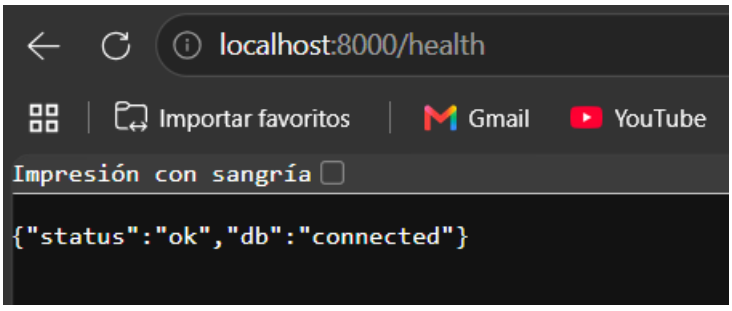
• Inyección de Carga: Grafana k6 operando localmente para inyectar tráfico HTTP simulando usuarios virtuales (VUs).

3. Pruebas de Integración y Despliegue Base

Antes de someter el sistema a cargas de estrés, se validó la correcta integración de los componentes y la conectividad entre la API y la base de datos de persistencia en dos fases clave.

3.1 Fase Local (Docker Compose)

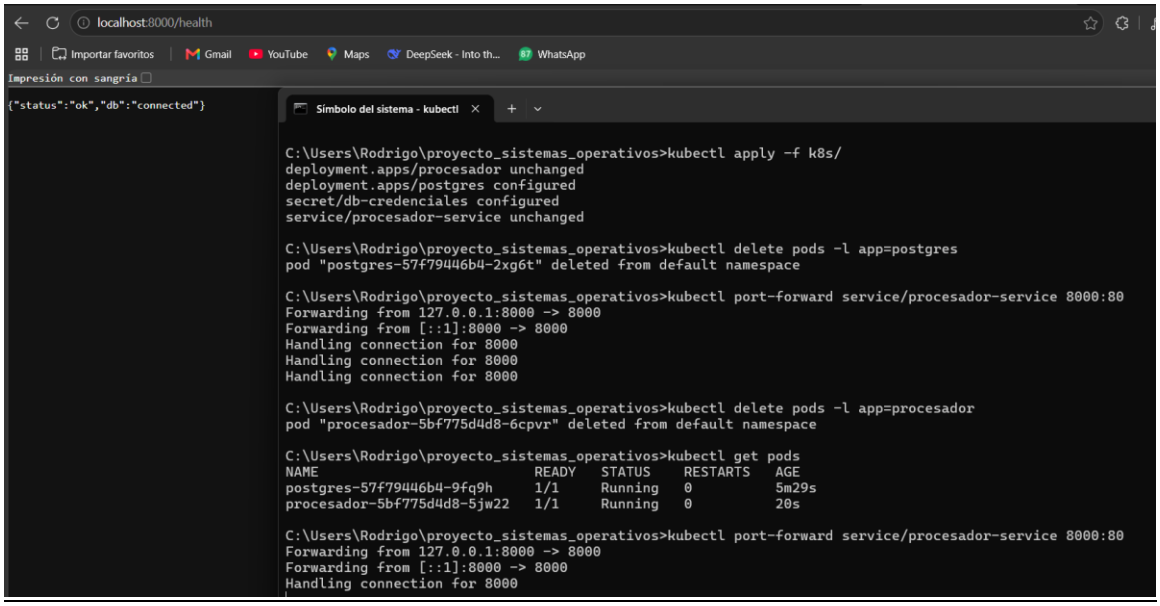
Se comprobó el correcto funcionamiento del pool de conexiones de psycopg2 en la API de Python conectándose a una instancia aislada de PostgreSQL mediante Docker Compose. El endpoint /health devolvió el estatus esperado.



```
#13 resolving provenance for metadata file
#13 DONE 0.0s
[+] up 3/3
 ✔Image proyecto_sistemas_operativos-procesador       Built                                    13.1s
 ✔Container proyecto_sistemas_operativos-db-1         Running                                   0.0s
 ✔Container proyecto_sistemas_operativos-procesador-1 Recreated                                 0.3s
Attaching to db-1, procesador-1
procesador-1  | INFO:     Started server process [1]
procesador-1  | INFO:     Waiting for application startup.
procesador-1  | INFO:     Application startup complete.
procesador-1  | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
procesador-1  | INFO:     172.27.0.1:48538 - "GET /health HTTP/1.1" 200 OK
procesador-1  | INFO:     172.27.0.1:48538 - "GET /favicon.ico HTTP/1.1" 404 Not Found
db-1          | 2026-11-11 04:34:20.098 UTC [55] LOG:  checkpoint starting: time
db-1          | 2026-11-11 04:34:24.449 UTC [55] LOG:  checkpoint complete: wrote 46 buffers (0.3%); 0 WAL file(s) added
, 0 removed, 0 recycled; write=4.321 s, sync=0.015 s, total=4.352 s; sync files=14, longest=0.011 s, average=0.002 s; di
stance=260 kB, estimate=260 kB
```

3.2 Fase de Orquestación (Minikube)

Tras contenerizar la aplicación, los manifiestos YAML fueron desplegados en Minikube. Se configuró exitosamente un ConfigMap para el script de inicialización SQL y un Secret para las credenciales de PostgreSQL. A través de un túnel port-forward (ClusterIP Service), se validó el acceso externo a la API desde el host físico.



4. Resultados Experimentales y Análisis de Métricas

A continuación, se desglosan los hallazgos de cada uno de los cinco escenarios (E1 a E5) definidos en la propuesta del proyecto, sustentados por las salidas directas de k6.

4.1 Escenario E1: Línea Base

Configuración: 1 Réplica | Carga Moderada (Max 10 VUs) | Duración: 50s

Resultados Obtenidos:

- Rendimiento (req/s): 7.76 solicitudes completadas por segundo.
- Latencia Mediana: 56.50 ms
- Latencia p(95): 83.48 ms
- Tasa de Error: 0.00% (389 solicitudes exitosas de 389).

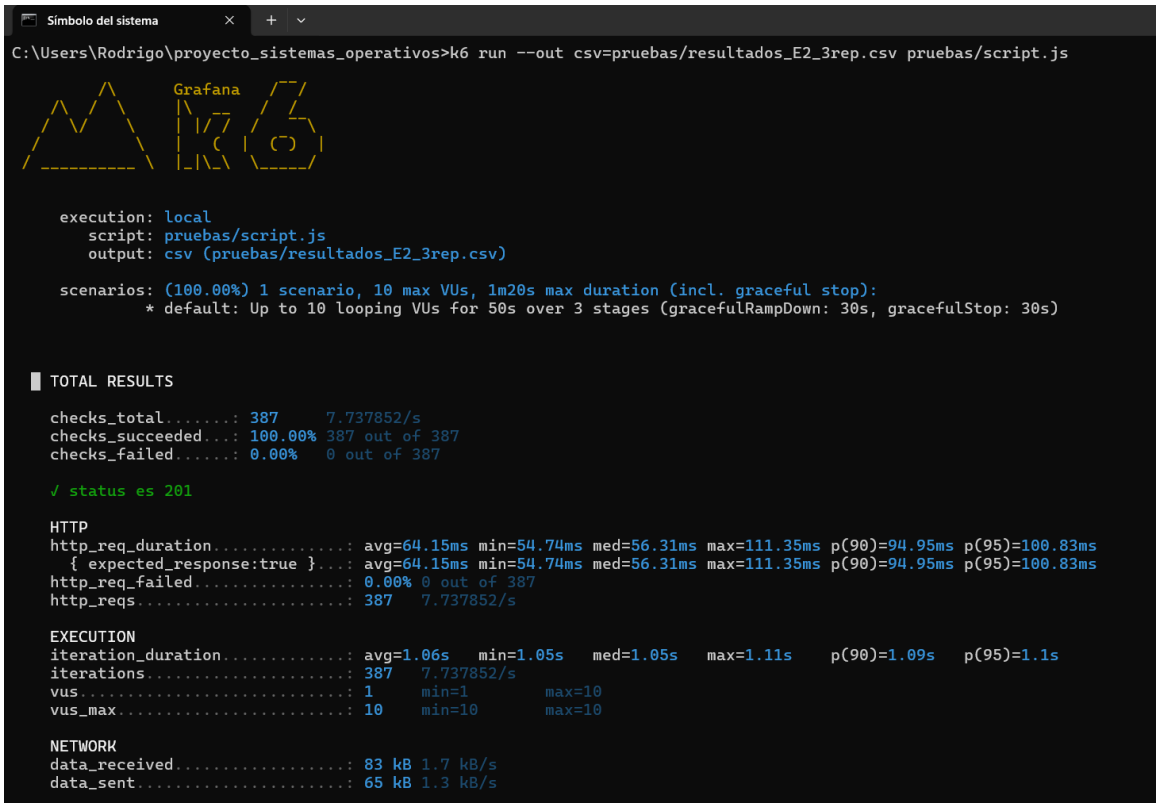
Análisis: El sistema establece una línea base sólida y estable. La latencia p95 por debajo de los 100ms indica que una sola réplica es perfectamente capaz de manejar concurrencia ligera (10 usuarios virtuales) sin bloqueos en el pool de conexiones.



4.2 Escenario E2: Escalabilidad Horizontal

Configuración: Comparativa 1, 2 y 3 Réplicas | Carga Moderada (Max 10 VUs)
Resultados con 2 Réplicas:
• Rendimiento: 7.61 req/s | Latencia p(95): 107.42 ms | Tasa de Error: 0.00%
Resultados con 3 Réplicas:
• Rendimiento: 7.74 req/s | Latencia p(95): 100.83 ms | Tasa de Error: 0.00%

Análisis de Escalabilidad: Aplicando la fórmula de mejora M(%) = ((R_n - R_1) / R_1) * 100, observamos que M(%) = ((7.74 - 7.76) / 7.76) * 100 = -0.25%. Conclusión Crítica: Escalar el número de réplicas NO mejoró el rendimiento bajo una carga moderada (10 VUs). De hecho, la latencia p95 sufrió un ligero incremento (de 83.48ms a ~100ms). Esto ocurre porque 10 usuarios no son suficientes para saturar 1 réplica. Al añadir más réplicas sin aumentar el tráfico, el sistema gasta más recursos en el balanceo de carga (Service de Kubernetes) que en procesamiento real.

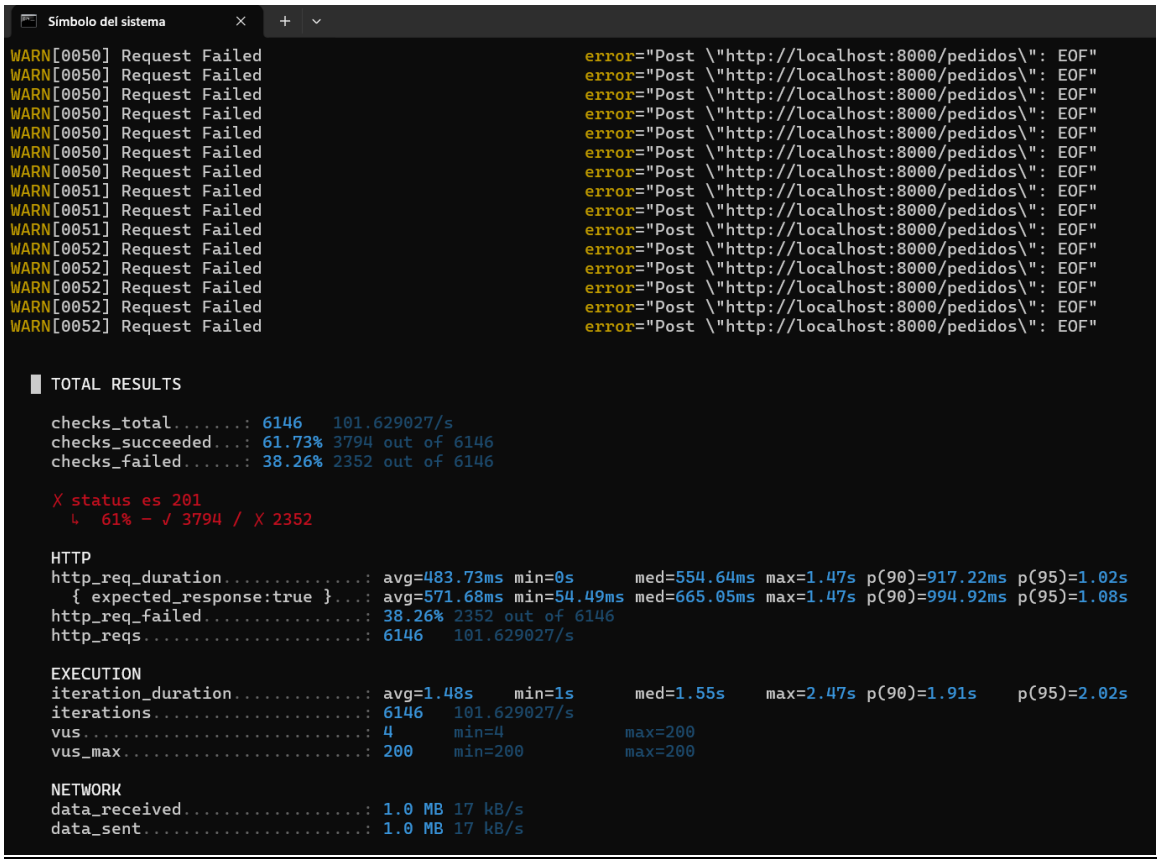


4.3 Escenario E3: Saturación y Alta Concurrencia

Configuración: 3 Réplicas | Carga Agresiva (Max 200 VUs) | Duración: 1m
Resultados Obtenidos:

- Rendimiento: 101.63 req/s (6146 peticiones en total)
- Latencia Mediana: 554.64 ms
- Latencia p(95): 1.02 s
- Tasa de Error: 38.26% (2352 fallos por EOF y Timeouts).

Análisis: Al someter el sistema a 200 usuarios concurrentes, se alcanzó el límite físico del diseño actual. La latencia p95 se degradó en un factor de 10 (pasando de 100ms a más de 1 segundo). El 38.26% de errores se derivó principalmente del agotamiento del Pool de Conexiones de la base de datos. Las réplicas de Python aceptan la conexión web, pero colapsan esperando que PostgreSQL libere un socket.



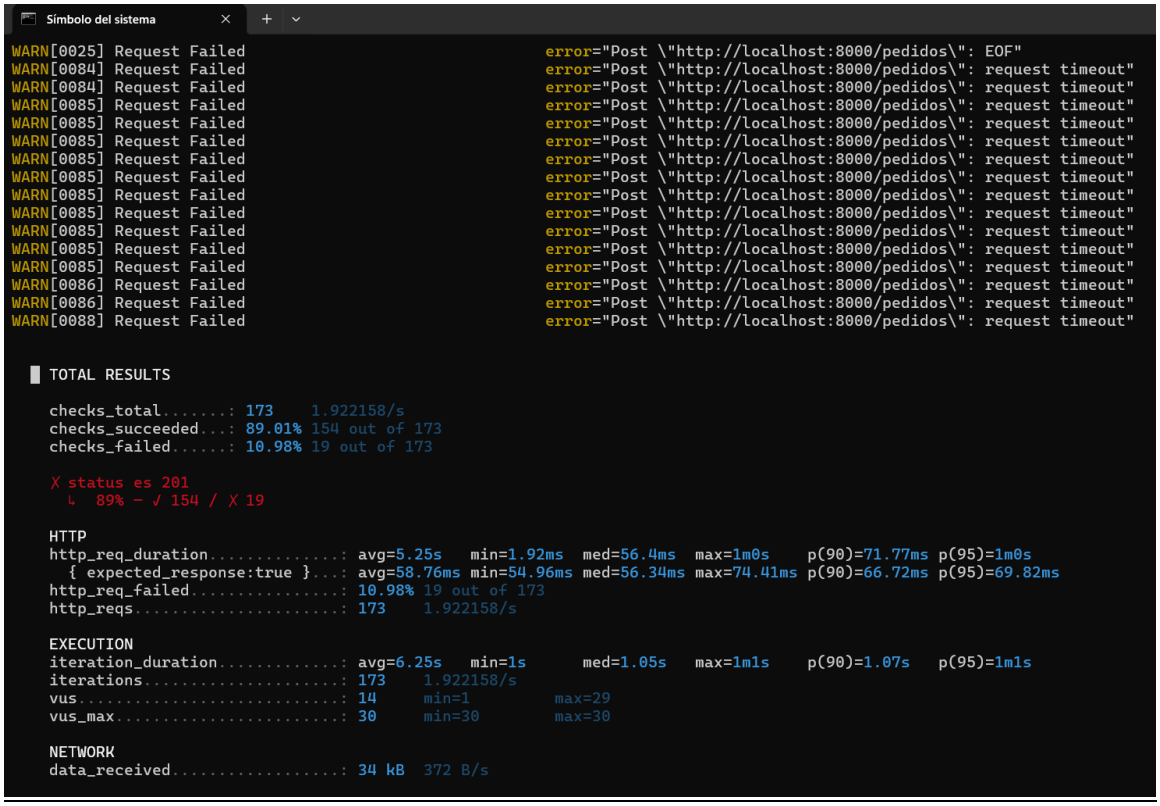
4.4 Escenario E4: Fallo de Instancia en Vivo

Configuración: 30 VUs constantes | Eliminación manual del Pod de PostgreSQL.

Resultados Obtenidos:

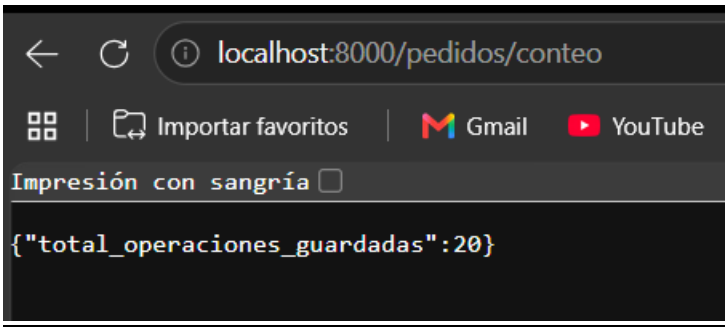
- Rendimiento: Caída a 1.92 req/s
- Tasa de Error: 10.98% (Timeout requests)
- Latencia p(95): 1m 0s (Límite de espera de k6)

Análisis: La inyección de la falla (borrado del pod) demostró la naturaleza de autorrecuperación de Kubernetes. El 10.98% de fallos documenta exactamente las peticiones "en vuelo" que se perdieron durante el lapso de inactividad. El controlador de Kubernetes detectó la anomalía y provisionó un nuevo pod de base de datos en pocos segundos, reestableciendo el servicio sin intervención humana adicional.



4.5 Escenario E5: Persistencia de Datos y Limitaciones Críticas

Se comprobó el endpoint de conteo de peticiones posterior a la recuperación del fallo. Resultó en únicamente 20 operaciones guardadas, validando un fallo estructural en el diseño inicial: La falta de Volúmenes Persistentes (Persistent Volumes - PV). Al eliminar el pod de PostgreSQL, el almacenamiento efímero interno fue destruido junto con él, provocando la pérdida de más de 6,000 registros insertados en las pruebas previas.



5. Conclusiones Finales

La evaluación experimental confirma los fundamentos teóricos de los sistemas distribuidos. A pesar de contar con un orquestador robusto como Kubernetes para mitigar las caídas de servicio, la escalabilidad de la capa de cómputo (FastAPI) está inherentemente ligada a la capacidad de la capa de persistencia.

Para entornos de producción reales basados en este experimento, es mandatorio:

- 1. Desacoplar el almacenamiento montando un Persistent Volume Claim (PVC) para PostgreSQL.
- 2. Implementar un gestor de conexiones intermedio como PgBouncer para mitigar los errores de agotamiento de pool observados en el escenario E3.
- 3. Configurar un Horizontal Pod Autoscaler (HPA) para que el clúster escale dinámicamente en base a picos de CPU, y no de forma estática.